PYTHON PROGRAMMING PROJECT

E-Commerce Customer Behaviour Analysis

Project Report and Case Study

Submitted by	Harkirat Singh Akshdeep Singh Rasanpreet Singh Sahibjot Singh Amitpal
University ID	2 7(B)10163 2 10203 24BCA10435 24BCA10159 24BCA10133
Section	7(B)
Subject Name	Python Programming
Subject Code	24CAH-301
Submitted to	Shobit Mahajan

DATA ANALYSIS PROJECT

Index

Combined project report and case study contents.

Page	Section
1	Cover page
2	Index
3	Abstract and introduction
4	Project objectives, scope, and requirements
5	Dataset design and assumptions
6	Data preparation and exploration
7	Correlation analysis
8	Customer clustering
9	Apriori association mining
10	Python implementation and design
11	Running the project and reading outputs
12	Case study context and questions
13	Case study analysis and findings framework
14	Recommendations, limitations, and ethics
15	Conclusion, references, and viva questions

Note: The included sample data is synthetic. Replace it with a real, documented dataset before presenting empirical conclusions.

1. Abstract and Introduction

This project uses Python to examine e-commerce order-line data from three complementary perspectives: numeric relationships, customer profiles, and product co-occurrence. It calculates Pearson correlation with purchase quantity, clusters customer summaries using standardized Euclidean k-means, and mines frequent product combinations using Apriori. Results are saved as CSV files and a readable text summary.

The included program uses the Python standard library and accepts a CSV file with one row per product line in an order. A synthetic sample dataset is supplied to demonstrate the workflow. It is illustrative and does not represent real retailer or customer behaviour.

Why these methods

Correlation is a compact way to screen linear associations between measured attributes. Clustering groups customer summaries that are similar across selected features. Association rules represent products that recur together in a transaction. These methods answer different questions, and their outputs should not be treated as causal effects.

Project deliverables

- A command-line Python program with configurable input, output, cluster count, support, and confidence.

- A small synthetic example dataset and setup instructions.

- Exported correlation, cluster membership, association-rule files, and a text summary after a run.

- This combined report and case study explaining the workflow and its responsible interpretation.

Report organization

Pages 3 to 11 describe the software project and methods. Pages 12 to 14 apply the workflow to a business case framework. Page 15 closes with references and oral-review questions.

2. Objectives, Scope, and Requirements

Objectives

- Load and validate e-commerce line-item records from CSV.

- Calculate Pearson correlation of numeric attributes with Quantity.

- Summarize customer activity and assign clusters with k-means.

- Discover frequent itemsets and rules from order baskets using Apriori.

- Export results in formats that can be inspected in a spreadsheet.

Scope

The project is a classroom-scale descriptive analysis. It includes one input schema and a deterministic, educational implementation of k-means and Apriori. It does not make recommendations automatically, predict future purchases, or establish causal effects. The cluster feature set is deliberately limited to average price, average rating, and total quantity per customer.

Requirements

Requirement	Description
Runtime	Python 3
External packages	None; Python standard library only
Input	CSV with transaction, customer, product, category, price, rating, and quantity columns
Outputs	CSV analysis files and a UTF-8 text summary
Use case	Small educational datasets and method demonstration

Success criteria

A valid run should read the specified input, reject missing required columns or malformed numbers with a clear error, create the output directory, and save the four documented output files. Findings must be described as sample-specific and based on the selected features and thresholds.

3. Dataset Design and Assumptions

The expected input has one record per product line within an order. TransactionID defines basket membership; CustomerID defines which line items are aggregated into customer summaries. ProductID names the item in the basket, while Category provides descriptive context.

Field	Meaning	Use
transaction_id	Unique order or basket key	Group product lines for Apriori
customer_id	Customer key	Aggregate customer profile; not a distance feature
product_id	Product identifier	Basket item for Apriori
category	Product group	Context for interpretation
unit_price	Line-item product price	Numeric correlation and customer mean
rating	Product/customer rating value	Numeric correlation and customer mean
quantity	Units purchased in line	Quantity measure and customer total

Data assumptions

- Prices use the same currency and reflect a consistent definition such as unit price before or after discount.

- Ratings share one scale and enough records to be interpreted.

- Transaction and customer keys are stable, non-empty identifiers.

- Positive quantity represents a purchase; returns and cancellations require an explicit policy.

- The included sample file is synthetic, generated for demonstration, and should not be used for real-world claims.

Representation matters

The same flat table supports numeric and transaction views but requires different preparation. Customer clustering uses customer-level aggregates. Apriori groups product IDs by transaction. Treating every line as a separate basket would miss co-purchases; treating identifier codes as numbers would create meaningless distance.

4. Data Preparation and Exploration

Validation workflow

1. Confirm the CSV header contains all required fields and inspect row counts.

2. Parse unit_price, rating, and quantity as numeric values; identify blanks, malformed values, and impossible values.

3. Check duplicate order lines, product key consistency, returns, negative quantities, and missing customer or transaction IDs.

4. Inspect distributions, ranges, and potential outliers before comparing customers.

5. Record any excluded rows, substitutions, category filters, and transformation decisions.

Exploratory checks

Summarize numeric ranges and missingness for price, rating, and quantity. Compare these fields by product category and, when sample sizes allow, by time period. Large price ranges or heavily skewed purchase quantities can dominate means and influence cluster profiles.

Preparation choices in this program

The program validates required column names and converts the three numeric values to floating point. It reports an error when a row contains an invalid numeric value. It does not automatically impute missing values, detect outliers, reconcile currencies, or remove duplicate lines; those decisions should be made and documented before using consequential data.

Identifiers and scaling

Customer and transaction keys are used for grouping only. Clustering standardizes average price, average rating, and total quantity before distance calculations, preventing the numerical units of price from automatically dominating the distance. The synthetic example is suitable for demonstrating code flow, not data-cleaning policy.

5. Correlation Analysis

Pearson correlation measures the direction and strength of a linear relationship between two numeric variables. The project calculates a score between each numeric field and Quantity, then writes the result to correlations.csv. Quantity is included against itself as a transparent reference value of 1.0.

Interpretation

- A positive score means higher observed values tend to appear with higher quantity in the analyzed records.

- A negative score means higher values tend to appear with lower quantity.

- A value near zero indicates little linear association in that sample, though nonlinear patterns may remain.

- The score is sensitive to outliers and does not control for product category, promotion, stock, time, or customer mix.

Formula

For variables X and Y, Pearson r is the covariance of X and Y divided by the product of their standard deviations. The implementation calculates centered deviations and normalizes their cross-product. If either variable has no variation, the program returns zero to avoid division by zero.

Reporting template

“For [N] valid line items, the correlation of [attribute] with Quantity was [r]. This indicates [direction and approximate strength] in this sample. The result may reflect [category / promotion / time / customer] effects and does not show that the attribute causes quantity to change.”

Caution

The provided example data has no reported empirical conclusions in this report. Run the program on the intended dataset and use the values written to correlations.csv before completing a results statement.

6. Customer Clustering

Clustering groups records based on feature similarity. This project aggregates line items per customer to compute average price, average rating, total quantity, and line count. It uses the first three measurements to form customer clusters with k-means.

Euclidean distance and centroid updates

For a standardized record x and centroid μ, Euclidean distance is the square root of the sum of squared coordinate differences. Each record is assigned to its nearest centroid. The centroid is then updated by taking the mean of each feature for records assigned to that cluster. Assignment and update repeat until cluster assignments stabilize or the iteration limit is reached.

Implementation choices

- Feature standardization uses the dataset mean and population standard deviation; constant features use scale 1.

- Initial centroids are selected at random with a fixed seed for repeatability.

- The requested number of clusters is capped at the number of customers.

- Cluster IDs are arbitrary labels; a cluster number does not imply rank or value.

How to interpret

Review group sizes and compare centroid profiles in the original price, rating, and quantity units. Describe a cluster only after inspecting the means and checking whether the pattern is stable under reasonable changes in cluster count and sample. Do not interpret the group as a permanent customer identity.

Limitations

K-means tends to favor compact, roughly spherical groups. It can be affected by outliers, feature selection, and the chosen cluster count. It summarizes the selected measurements and omits many behaviours such as recency, category preference, and time between purchases.

7. Apriori Association Mining

Apriori searches transaction baskets for frequent itemsets and rules. Product IDs with positive quantity are grouped by transaction ID. Candidate itemsets are expanded only when their smaller subsets are frequent, using the Apriori property.

Key measures

Measure	Meaning
Support	Share of all baskets containing the itemset
Confidence	Share of baskets containing the antecedent that also contain the consequent
Lift	Confidence divided by the baseline frequency of the consequent; above 1 indicates positive co-occurrence relative to baseline

Thresholds

Minimum support controls how common an itemset must be. Minimum confidence filters rules by the proportion of antecedent baskets that also contain the consequent. Lower thresholds usually return more rules, including weak or rare patterns; high thresholds can hide useful niche combinations. Defaults are configurable from the command line.

Interpreting rules

Read support, confidence, and lift together. A rule with high confidence but very low support may apply to very few baskets. A lift above one can indicate a co-occurrence pattern relative to the consequent baseline, but it does not establish why the products appear together or whether promoting them will increase sales.

Practical checks

- Confirm that transaction_id represents a meaningful basket boundary.
- Check basket counts, item encoding, returns, and rare product frequency.
- Record both thresholds and total transaction count.
- Validate promising rules on another time period or in a controlled merchandising test.

8. Python Implementation and Design

The project is organized as a small command-line application. The source file contains data loading, correlation, customer aggregation, k-means, Apriori, CSV export, and argument parsing. The implementation uses csv, argparse, pathlib, collections, math, and random from the Python standard library.

Component	Responsibility
load_rows	Read CSV, check required headers, convert numeric columns
correlations / pearson	Compute feature-to-quantity Pearson scores
customer_features	Aggregate line-item measures per customer
kmeans	Standardize measurements and iteratively update assignments and centroids
apriori	Build transaction baskets, frequent itemsets, and confidence-filtered rules
run / write_csv	Save output files and a readable run summary
main	Parse command-line options and validate thresholds

Command-line options

`--input` selects the input CSV. `--output` selects the results folder. `--clusters` sets k. `--min-support` and `--min-confidence` set Apriori thresholds. Defaults point to the included sample and create a results directory in the project.

Educational implementation

The algorithms are written explicitly to make the workflow understandable. This design prioritizes clarity for small datasets over throughput, advanced optimization, or production-grade error handling. For larger data, use optimized libraries and compare their outputs against a clearly documented baseline.

9. Running the Project and Reading Outputs

Run with the sample data

Open a terminal in the project directory and run:

```
python src/ecommerce_analysis.py
```

Use a different dataset

```
python src/ecommerce_analysis.py --input data/my_orders.csv --output results
--clusters 3 --min-support 0.08 --min-confidence 0.25
```

The CSV must contain transaction_id, customer_id, product_id, category, unit_price, rating, and quantity. One row represents one product line.

Generated files

File	Contents
correlations.csv	Pearson scores by numeric attribute
customer_clusters.csv	Customer aggregates and cluster assignment
association_rules.csv	Antecedent, consequent, support, confidence, and lift
summary.txt	Input path, counts, correlations, cluster sizes, and leading rules

Review steps

1. Check the data path and row counts in the summary.

2. Inspect correlation values and confirm that quantity against itself is not treated as a discovery.

3. Compare cluster counts and profiles; revisit features or k if a cluster is empty or uninformative.

4. Sort association rules by lift and support, and manually review products and thresholds.

5. Save a copy of the source data, exact command, software version, and outputs with the final submission.

Do not present synthetic data outputs as business findings. Replace the sample with real, appropriate data and complete the case-study framework with measured results.

10. Case Study: Context and Questions

Business context

An online retailer wants a concise view of how product price and customer ratings appear alongside purchase quantity, whether customer profiles differ across selected purchasing measures, and which products recur in the same baskets. Managers hope the analysis can guide follow-up questions about product grouping and merchandising.

Problem statement

Order-line data is detailed but difficult to interpret directly. Numeric correlations, customer-level summaries, and transaction baskets each require a different analytical view. A combined workflow can organize the evidence, but it cannot by itself identify causal drivers or guarantee that a promotion will work.

Questions for analysis

- How does quantity correlate with price and rating in the observed line items?

- Can customers be grouped by average price, rating, and purchase volume?

- Which products occur together often enough to consider for a controlled merchandising test?

Case data

The project schema provides transaction IDs, customer IDs, product IDs, categories, prices, ratings, and quantities. A real case would also document the collection period, returns policy, discounts, stock availability, and whether each price is list or paid price. No real retailer dataset was supplied for this report; the included CSV is synthetic.

Stakeholders and decision context

Analysts prepare and validate the dataset; merchandising staff assess product context; decision makers determine whether an idea merits a small test. Customer data should be handled under the organization’s privacy and access rules, with direct identifiers excluded from reports unless they are necessary and authorized.

11. Case Study: Analysis and Findings Framework

Step 1: Prepare the evidence

Confirm the number of order lines, baskets, customers, and products. Check missing data, duplicates, returns, rating coverage, price consistency, and category mix. Record exclusions and transformations before running the methods.

Step 2: Review numeric relationships

Run the Pearson calculation against Quantity. Examine price and rating coefficients alongside distributions and category-specific views. If results differ sharply across categories or time, report the context rather than only the pooled coefficient.

Step 3: Describe customer groups

Aggregate to customer level and run k-means over the chosen standardized fields. Compare cluster sizes and centroid profiles. Test more than one k and seed where appropriate; use plain descriptions such as higher average price and lower order-line quantity only when supported by centroids.

Step 4: Review basket rules

Group items by transaction and run Apriori. Review support, confidence, and lift with the raw basket count. Check whether a high-lift rule is based on sufficient transactions and whether the items are complements, substitutes, or routinely sold together for contextual reasons.

Findings table to complete after analysis

Question	Observed evidence to insert
Price and quantity	Coefficient, valid N, category/time caveat
Rating and quantity	Coefficient, rating coverage, caveat
Customer groups	k, sizes, original-unit centroid profiles
Product combinations	Rule, support, confidence, lift, basket count

This report intentionally does not invent measured findings. Enter the values from the actual program run and identify the dataset and analysis period.

12. Recommendations, Limitations, and Ethics

Recommendations

- Use supported basket rules to nominate a bundle or placement idea for a controlled test.

- Compare price and quantity within relevant product categories before suggesting price changes.

- Treat customer clusters as temporary analytical summaries and verify they are stable and useful before tailoring outreach.

- Track outcomes such as conversion, margin, returns, and customer experience in a later time period or experiment.

- Keep analysis settings, data definitions, and output files so another reader can reproduce the result.

Limitations

- Pearson correlation captures linear association and can be distorted by outliers or confounding variables.

- K-means depends on feature choice, scaling, initialization, and cluster count.

- Apriori results depend on basket definition and support/confidence thresholds.

- Repeated customers, seasonality, discounts, stockouts, and selection effects can shape observed patterns.

- The bundled example is synthetic and cannot support statements about an actual retailer.

Responsible use

Use only data that is appropriate and authorized for the assignment. Minimize personal information in shared outputs. Avoid exposing identifiable customer records. Communicate uncertainty and do not use a discovered pattern as a guarantee of future behaviour. If a business action affects customer treatment, evaluate fairness, privacy, and unintended effects before deployment.

Validation plan

A practical next step is to repeat the analysis on a later period, compare stable patterns, and test promising ideas in a small controlled experiment. Record success measures and guardrails in advance.

13. Conclusion, References, and Viva Questions

This Python project demonstrates a compact analytical workflow for e-commerce line-item data. Pearson correlation screens numeric relationships with quantity; standardized Euclidean k-means groups customer summaries; and Apriori identifies recurring product combinations in transaction baskets. The separate data representations and cautious interpretation are essential to making the outputs useful.

The project is immediately runnable with the included synthetic sample. Company-specific conclusions require an actual, cleaned dataset and transparent reporting of data period, definitions, thresholds, and results.

References

- Python Software Foundation. Python 3 Standard Library documentation: csv, argparse, pathlib, collections, math, and random.

- Agrawal, R., Imieli■ski, T., and Swami, A. “Mining Association Rules Between Sets of Items in Large Databases.” ACM SIGMOD, 1993.

- MacQueen, J. “Some Methods for Classification and Analysis of Multivariate Observations.” Proceedings of the Fifth Berkeley Symposium, 1967.

Viva questions

- Why should customer and transaction identifiers not be treated as numeric distance features?

- Why does standardization matter for Euclidean clustering?

- How do support, confidence, and lift differ?

- Why does a correlation or association rule not establish causation?

- How would you test whether a discovered pattern is stable over time?